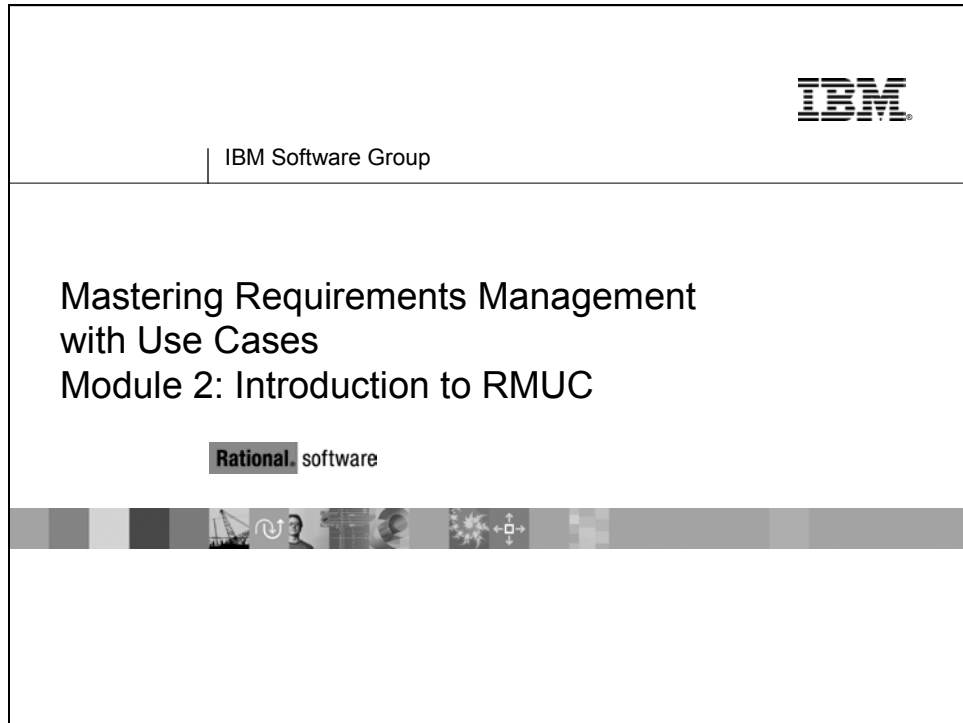


► ► ► Module 2 Introduction to RMUC



Topics

Definitions	2-5
Effective Requirements Management	2-13
What is a "Quality Product" ?	2-14
What Factors Contribute to Project Success?	2-19
The High Cost of Requirement Errors.....	2-21
Help Projects Succeed.....	2-22
Involve the Whole Team in Requirements	2-23
What Is NOT in a Requirement?.....	2-34
Requirements Discipline: Workflow Details	2-36

Objectives

Objectives
♦ Define key requirements management terms. ♦ Identify contributing factors to project success and project failure. ▪ Describe how requirements management increases the chances of project success. ♦ Describe qualities of requirements sets. ▪ Verifiable, traceable, unambiguous. ♦ Describe the RUP® requirements management workflow, roles, and artifacts.
2

Some Familiar Situations...

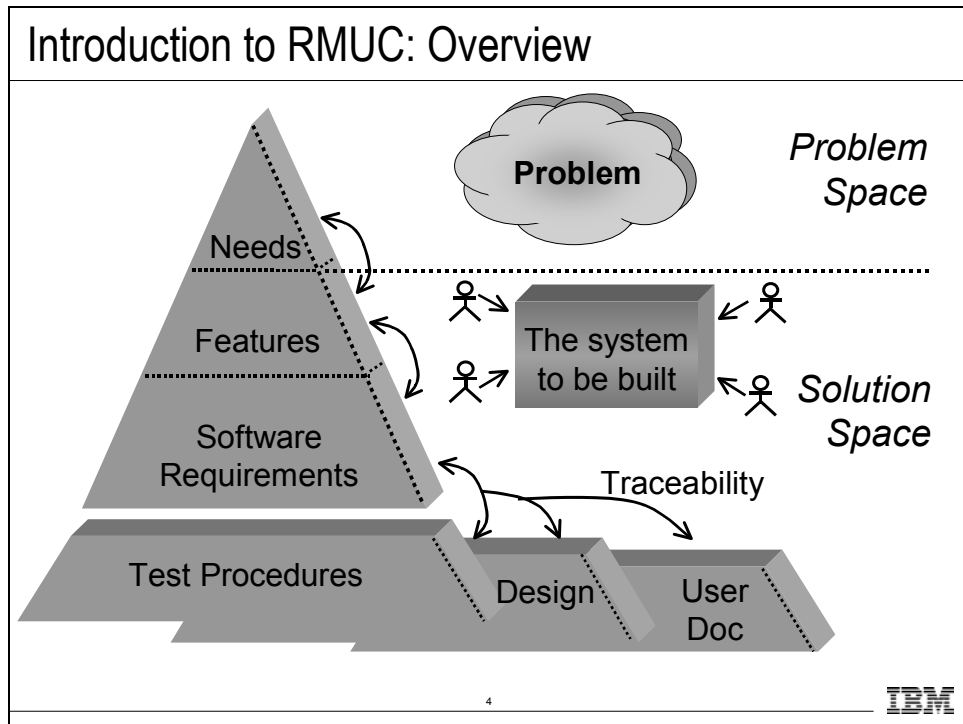


Most people have experienced project failures of one form or another, and there are probably lots of reasons why the failures happened. As most of you know, the software industry has much room for improvement when it comes to delivering success.

The four cartoons show symptoms of common project problems:

- Top left cartoon: Skipped analyzing the problem or understanding their stakeholders' needs.
- Top right cartoon: Poor elicitation of requirements, lack of stakeholder involvement.
- Bottom left: Developers think they know best, requirements not kept formally, no formal change management.
- Bottom right: Failure to manage scope.

Introduction to RMUC: Overview



Managing requirements involves the translation of stakeholder requests into a set of system features. These in turn are detailed into specifications for functional and nonfunctional requirements. Detailed specifications are translated into test procedures, design, and user documentation.

Definitions

Definitions
♦ Requirement ▪ A condition or capability to which the system must conform. ♦ Requirements management ▪ A systematic approach to: • Eliciting, organizing, and documenting requirements. • Establishing and maintaining agreement between customer/user and the project team on the changing requirements.
5 

Definitions:

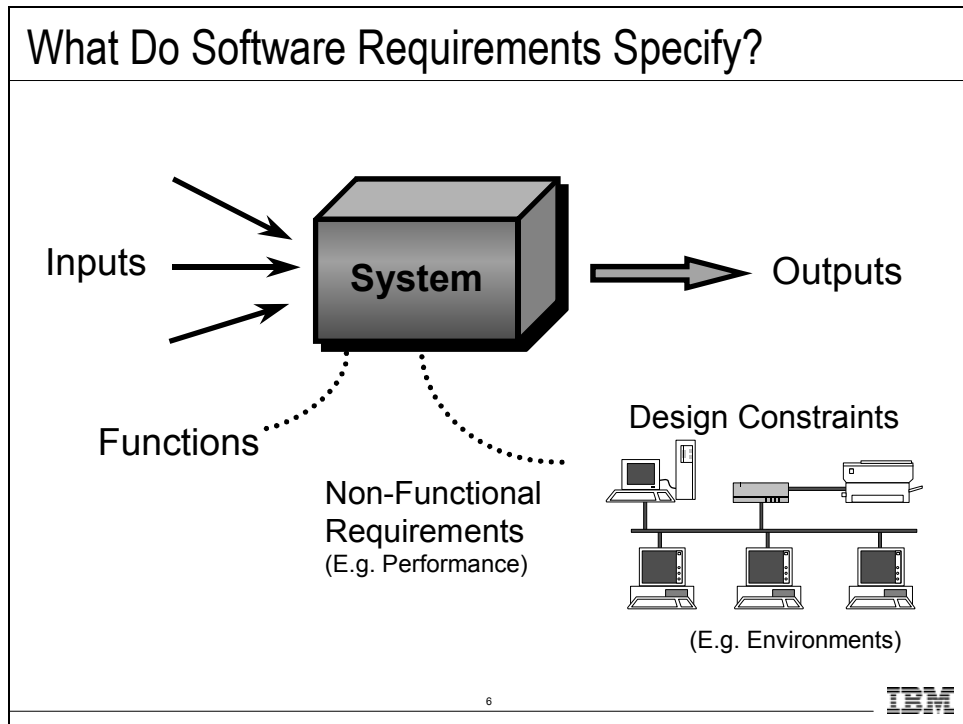
- **RUP:** A requirement describes a condition or capability to which a system must conform, either derived directly from user needs, or stated in a contract, standard, specification, or other formally imposed document.
- **UML:** A desired feature, property, or behavior of the system.

Requirements specify what the system must do rather than how the system does it.

There are many kinds of requirements. For example, *feature* requirements are requirements that are directly tied to user needs, and *software* requirements give the detailed requirements for the software. These different types of requirements are discussed later in the course.

Do not expect to “get it right first time”. Requirements management is successful only if it allows for uncertainty of the requirements early in the project. However, requirements management also ensures that requirements become better defined over time.

What Do Software Requirements Specify?



A requirement is a capability or condition to which the system must conform.

Software requirements provide a “black box” definition of the system. They define only those externally observable “What’s” of the system, not the “How’s.”

Of course, there has to be enough of the “How” specified to build the right system, but these should be identified as design constraints.

Definitions

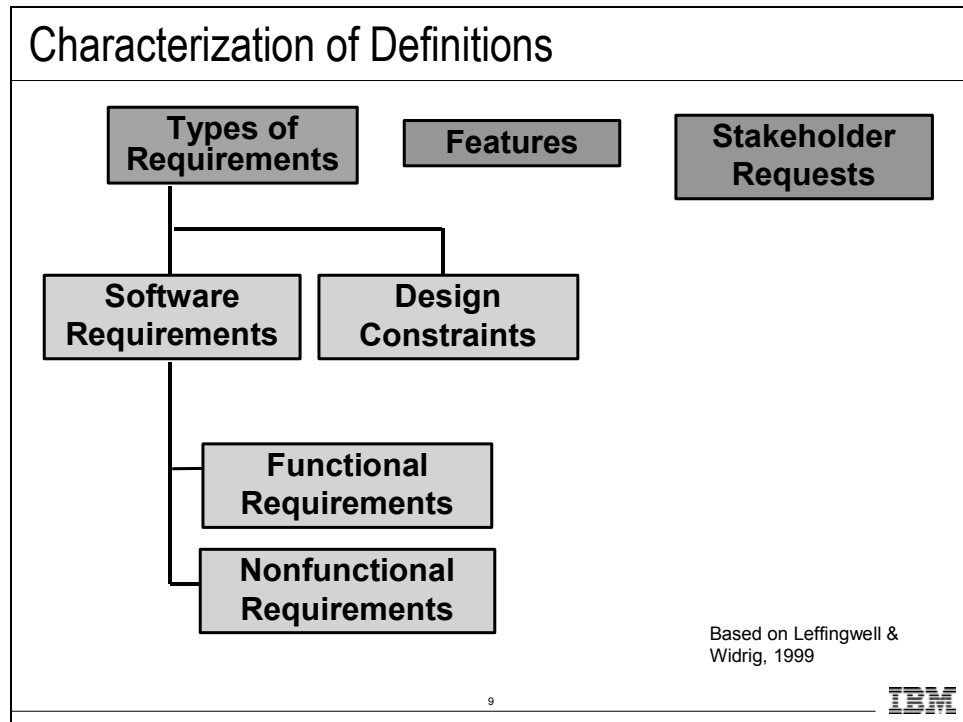
Definitions	
Stakeholder Request	A solution-independent expression of a stakeholder's desired state in the solution or subject domain.
Feature	An externally observable service by which the system directly fulfills one or more stakeholder requests.
Software Requirements	
<u>Functional Requirement</u>	A requirement that specifies, from a black-box perspective, how the solution interacts with the outside world.
<u>Non-Functional Requirement</u>	A requirement that expresses, from a black-box perspective, the quality attributes of the solution.
Constraint	A limitation on the design of the system or the process used to build the system.
7 	

There is some overloading of terminology between Stakeholder Request and Stakeholder Need. In this course, stakeholder needs and stakeholder requests are essentially synonymous. Stakeholders have certain needs of a solution. Those needs are expressed as stakeholder requests.

Examples from the Course Registration System

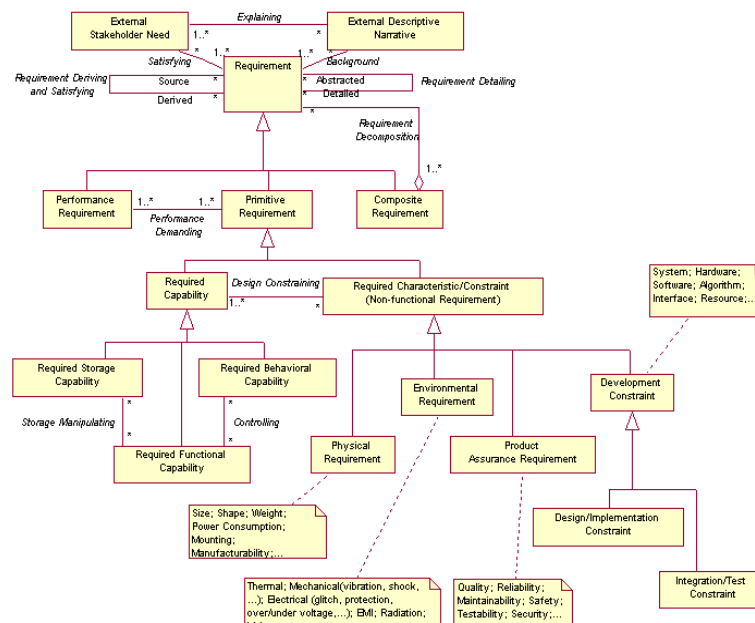
Examples from the Course Registration System	
Stakeholder Request	▪ Need less administrative overhead for registration. ▪ Professors need immediate access to student grades.
Feature	A tree browser provides a way to view student information, by semester by class.
Software Requirement	<u>Functional</u> The use case starts when the student selects the “register for course” command. The system displays the list of available courses... <u>Non-Functional</u> 99% of 24/7 availability (3.65 days downtime per year)
Constraint	Operate on the College DEC VAX Main Frame.

Characterization of Definitions

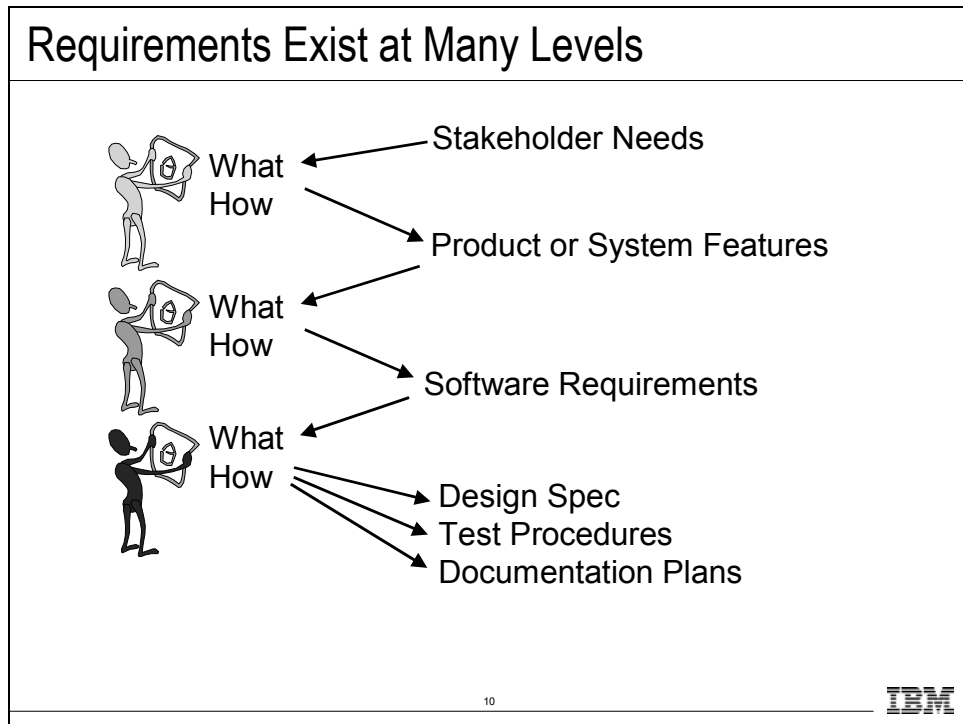


Categorizing the requirements helps determine if you have found all the requirements. For example, ask “are there any constraints on the system we have not identified yet?” or “What are the performance requirements of this system?”

The following more exhaustive taxonomy is based on: "Process for System Architecture and Requirements Engineering" by Derek Hatley, Peter Hruschka, and Imtiaz Pirbhai (Dorset House Publishing).



Requirements Exist at Many Levels



Requirements exist at many levels. Depending upon your perspective, a specification may be a requirement or a design.

For example, a stakeholder's need is a requirement on the system analysts. The requirements analysts produce a list of features that are requirements on the use-case specifiers. The use cases they write must provide a specification of what the system does to implement those features. The use-case specifiers produce a number of use cases that are requirements on the designers. This pattern continues down the chain.

In order to produce the correct system solution description, it is important that there be no misunderstanding as to what the needs of the stakeholders are. Therefore, the qualities of requirements play as much a part when capturing stakeholder needs as they do when you write the detailed software requirements.

Requirements Management Is Not Easy Because...

Requirements Management Is Not Easy Because...

- ♦ Requirements:
 - Are not always obvious.
 - Come from many sources.
 - May not always be easy to express clearly in words.
 - Relate to one another and to other deliverables of the software engineering process.
 - Have unique properties or property values.
 - Change.
 - Are difficult to control in large numbers.

11

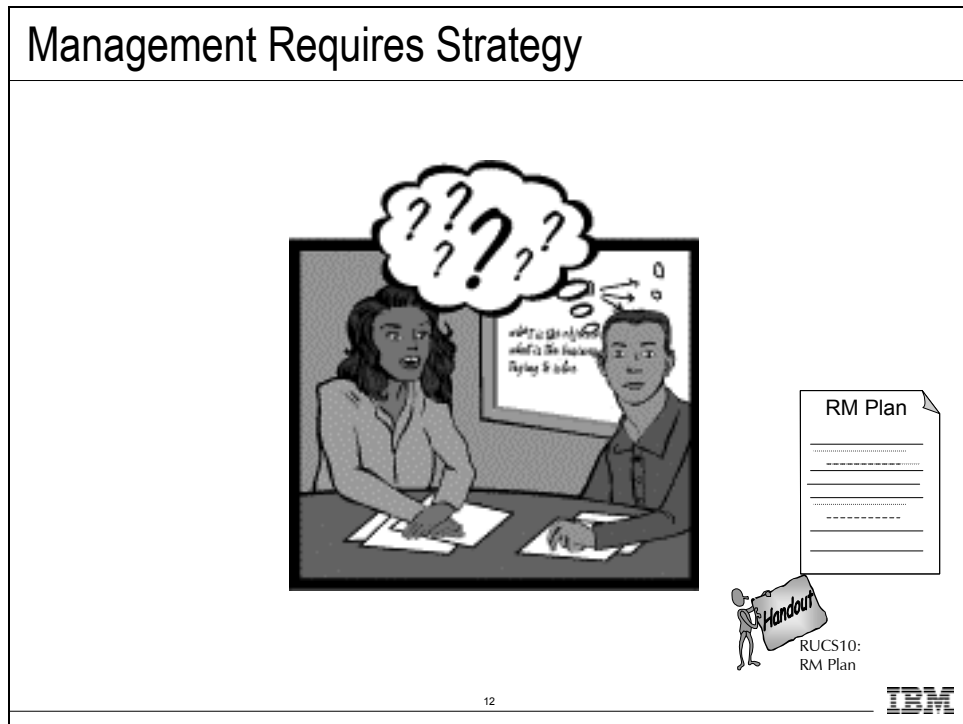


Requirements management is simple in theory, but difficult in practice. There are many reasons for this. Some include:

- Customers do not always know what they want.
- As the number of requirements grows, your ability to keep a handle on them all decreases.
- Relationships between requirements is not easily managed.
- People in the IT industry generally have trouble writing in a style that is understandable to people who are not as technically inclined.

These challenges can be overcome with the right processes in place.

Management Requires Strategy



A Requirements Management Plan (RM Plan) should be developed to specify the information and control mechanisms that are collected and used for measuring, reporting, and controlling changes to the product requirements.

Before you start to describe the project requirements, you must decide how to document and organize them. As well, you must decide how to use requirements attributes when managing the requirements throughout the project lifecycle.

The RM Plan documents all decisions regarding requirements documents, requirement types, guidelines and strategies for requirement attributes, and requirement traceability.

As with the Glossary, the RM Plan is a living document. You should start developing it as soon as the project is begun. You add to it as more decisions are made about attributes and traceability. In the activity called "Manage Dependencies," you further develop the project.

Ideally, your organization should have a standard RM Plan that you can adopt and customize for each project. This helps speed up the Inception phase of the project.

What is in a Requirements Management Plan?

- Types of requirements to collect and where you will collect them.
- Types of attributes to track.
- Types of requirements to trace.
- Types of documents to produce.
- Management guidelines.

Effective Requirements Management

Effective Requirements Management

- ♦ Maintain a clear statement of the requirements with:
 - Good quality requirements
 - Applicable attributes for each requirement type
 - Traceability to other requirements and other project artifacts

The GOAL is to deliver ***quality*** products
on time and on budget
that meet the customer's
real needs.

13



In the bigger picture, why do you care about requirements management?

Over the next few pages, you will explore what is meant by quality, on time and on budget, and real needs.

What is a “Quality Product” ?

What is a “Quality Product” ?

♦ Quality: *Old Concepts*

- Satisfies requirements documents.
 - Passed system test.
 - Development adhered to process.
- Activity-based paradigm

♦ Quality: *Modern Concept*

- Understand all stakeholder needs.
 - Continually assess all artifacts to see if needs are met.
- Results-based paradigm

14



Software development has traditionally been focused on delivering a system that implements a complete requirements specification. The old definition of a quality system was when "the system was delivered as specified", which of course does not ensure the system's success in the stakeholders application environment (Return on Investment). This understanding of quality is consistent with an activity management paradigm. That is, you start your project with a detailed project plan following a detailed process. This involves a leap of faith that the process, if followed, will result in quality. Because there was no "thing" to "touch and feel" as the project progressed, reviewing requirements specifications to assess whether the system would really solve the business problem was a subjective process rather than an objective process. System tests were used to verify that the system was correct instead of having the users accept that it helped solve their problem. The process for the development of the system consists of distinct activities, such as requirements, analysis, design, and test.

While adhering to process is important, it is important to understand the dimensions of software quality and how it is achieved. The focus then is on the results. Thus, modern software development practices are more focused on understanding what the stakeholders require from a system in order to solve a business problem. The iterative nature of modern software development practices also ensures that there is a continual review and feedback cycle that can refine the requirements as the project progresses. By delivering functional executable versions of the final product at the end of each iteration, the stakeholders can assess and refine their needs objectively instead of subjectively. The focus is therefore, not focusing on the activities, but on achieving results.

Dimensions of Quality

Dimensions of Quality		
Components of FURPS+		
F unctionality	Feature set capabilities, security, generality	
U sability	Human factors aesthetics, consistency, documentation	
R eliability	Frequency/severity of failure, recoverability, predictability, accuracy, MTBF	
P erformance	Speed efficiency, resource usage, throughput, response time	
S upportability	Testability Adaptability Compatibility Serviceability Localizability	Extensibility Maintainability Configurability Installability Robustness

Grady, 1992

15



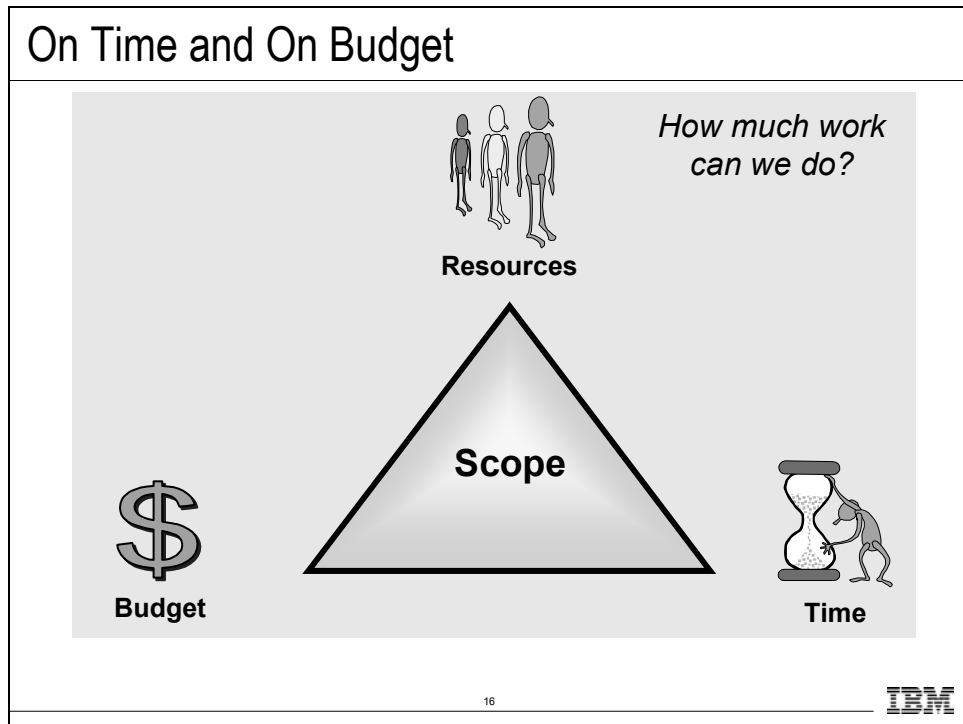
The FURPS acronym represents a portion of the requirement types for which you should be searching; it reminds us to get nonfunctional (URPS), as well as functional (behavioral) requirements.

The “+” after the FURPS indicates constraints that do not fit into any of these categories.

Can you think of some other examples of the types of requirements to look for?

Note: MTBF = Mean Time Between Failures

On Time and On Budget



Because your time and resources (people, money, equipment, and so on) are limited, you can complete only a fixed amount of work.

To deliver a product “on time and on budget”, you need to figure out how much work you can actually complete with a given scope, budget, and pool of resources.

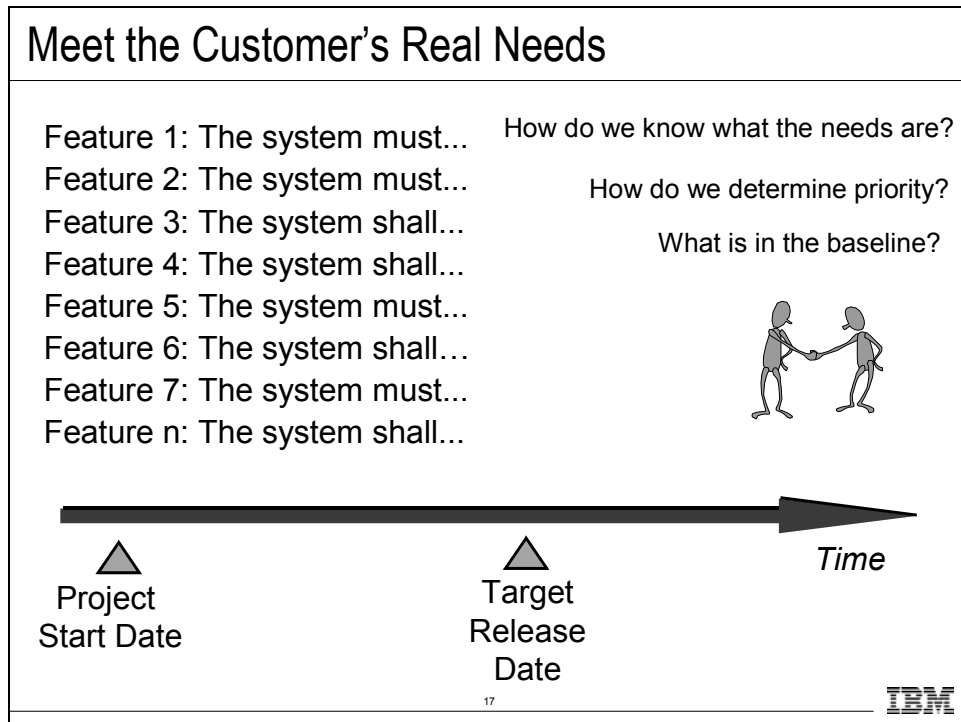
If any one of the four factors change, you need to make adjustments.

For example, if you increase scope, you need to increase one or more of the following: time, money and resources. If you reduce the budget, you need to decrease the scope to deliver the system in the same time and with the same resources.

Failure occurs if you keep squeezing more and more into the triangle without moving one of these points to compensate.

Of course you want to deliver the maximum value, so be as accurate as possible.

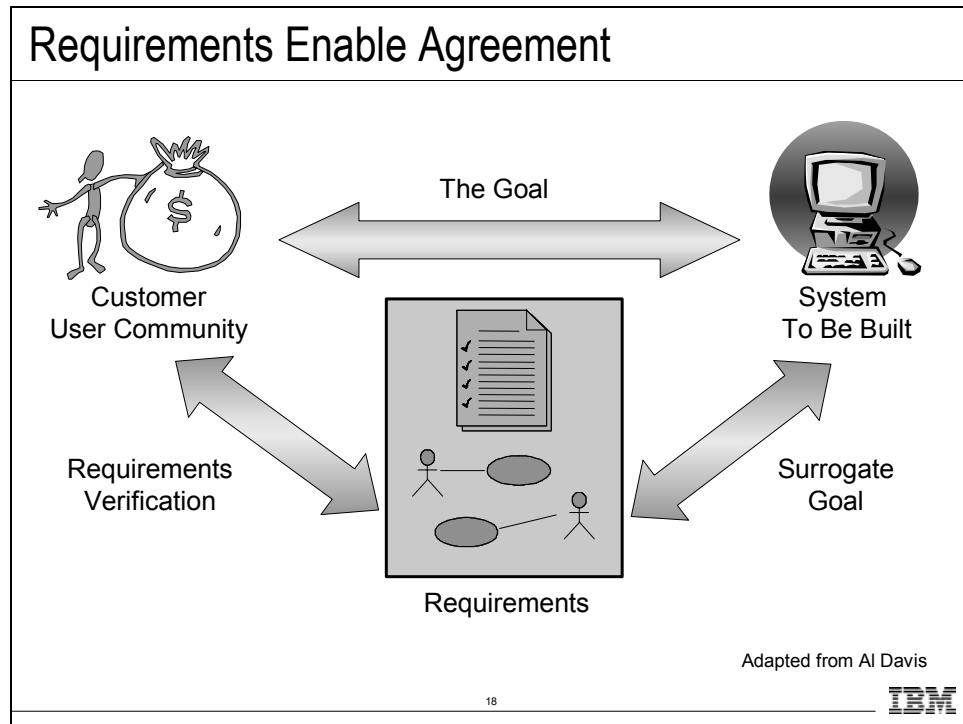
Meet the Customer's Real Needs



How do you reach agreement on what features should be included in our project?
 What stakeholder needs do they represent? How should they be prioritized?

What should you put in the baseline commitment for delivery? “Baseline” means the set of features that constitutes the agreed-upon basis for development and that can only be changed through a formal procedure.

Requirements Enable Agreement



One of the goals of requirements management is to come to agreement on what the system should do. This agreement is expressed in the requirements specification.

Because you probably do not have direct access to your customer and user community at all times and because some customers have been known to change their minds, it is important to write down the agreed-upon requirements of the system to be built.

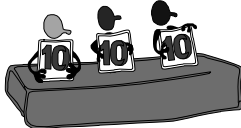
The requirements specification is a "proxy" for the customer. Your requirements specification should be complete enough so that you do not need to continually consult with the customer during implementation.

Requirements can be viewed as a "proxy" for the customer because the requirements represent the customer. That is, the requirements provide the details of the customer's desires and agreement on what the system should do.

The requirements should be captured in a form that is understandable to both the customer and the development team. The requirements provide the surrogate goal for the development team while building the system, as well as the criteria for acceptance and validation of the system by the customer upon delivery.

What Factors Contribute to Project Success?

What Factors Contribute to Project Success?



Project Success Factors

- **28%** of projects completed on time and on budget.
- **49%** of projects overran original estimates.
 - Time overrun averaged 63%.
 - Cost overrun averaged 45%.
- **23%** of projects canceled before completion.

The CHAOS Ten

- 1. Executive Management Support**
- 2. User Involvement**
- 3. Experienced Project Manager**
- 4. Clear Business Objectives**
- 5. Minimized Scope**
- 6. Standard Software Infrastructure**
- 7. Firm Basic Requirements**
- 8. Formal Methodology**
- 9. Reliable Estimates**
- 10. Other**

Standish Group, '01 (www.standishgroup.com)


Notice that numbers 2, 4, 5, 7, 8, and 9 are related to requirements management. These items are covered in this course.

The Standish Group surveys the IT industry every two years, beginning in 1994. Their Chaos Report provides insight into the success or failure of projects. The 2001 Chaos Report analyzed the historical data of past CHAOS reports and identified the latest factors for project success.

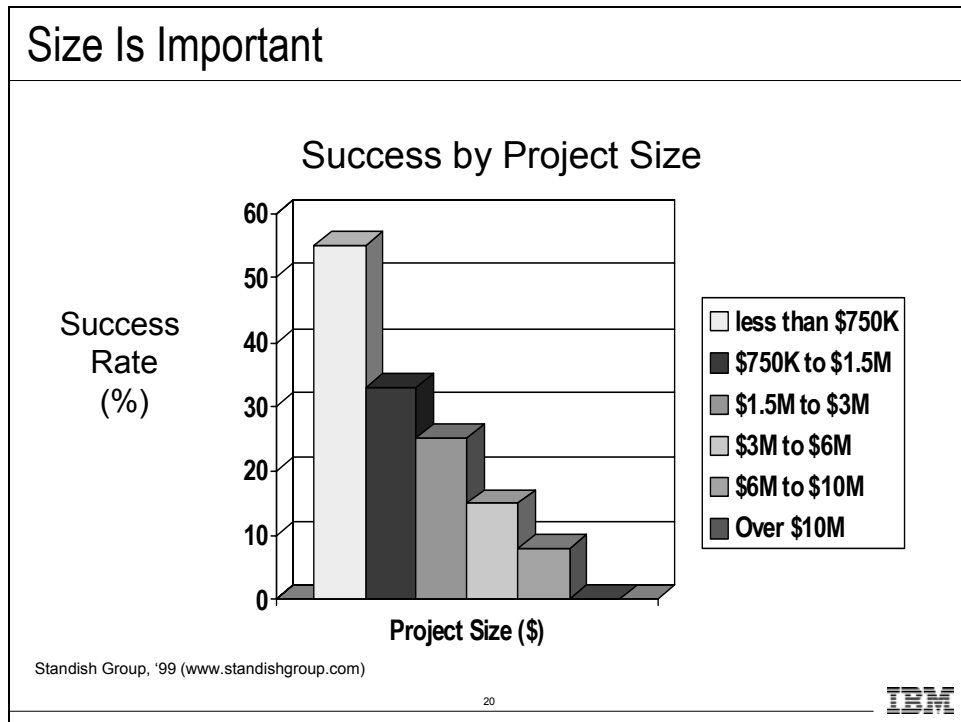
Comparing the results of surveys over previous years, the Standish Group concludes that there has been decided improvement in the IT field. Project success rates are up across the board 28% (overall success rate in 1994 was 16% and in 1998 it was 26%), while cost and time overruns were down. Despite this progress, The Standish Group research shows that challenged and failed projects remain the norm, and the cost of failed projects is staggering.

Executive management support has replaced user involvement for the first time.

According to the Standish Group: "Without a staunch project champion with a solid business vision, projects can drift into a technical or political abyss."

The CHAOS ingredients for the recipe for success are: "Reduce requirements to a bare minimum, providing constant communication systems and coupling those with a standard infrastructure. Mix these with good stakeholders, an iterative process, project management tools, adherence to key roles, and success is practically in the oven." The 2001 Chaos Report also reinforced that project success is positively correlated with small project size, short project duration, and small team size. Small projects are easier to manage and plan: "... no more than four people, for no longer than four months at a cost of less than \$500,000."

Size Is Important



The Standish Group believes that three key metrics can pinpoint a project's success potential: project size, team size, and project duration.

Although one school of thought would argue that larger projects with more funding and resources should be more successful and should produce more function, the reverse appears to be true. Smaller projects tend to be more manageable.

How does this relate to requirements management?

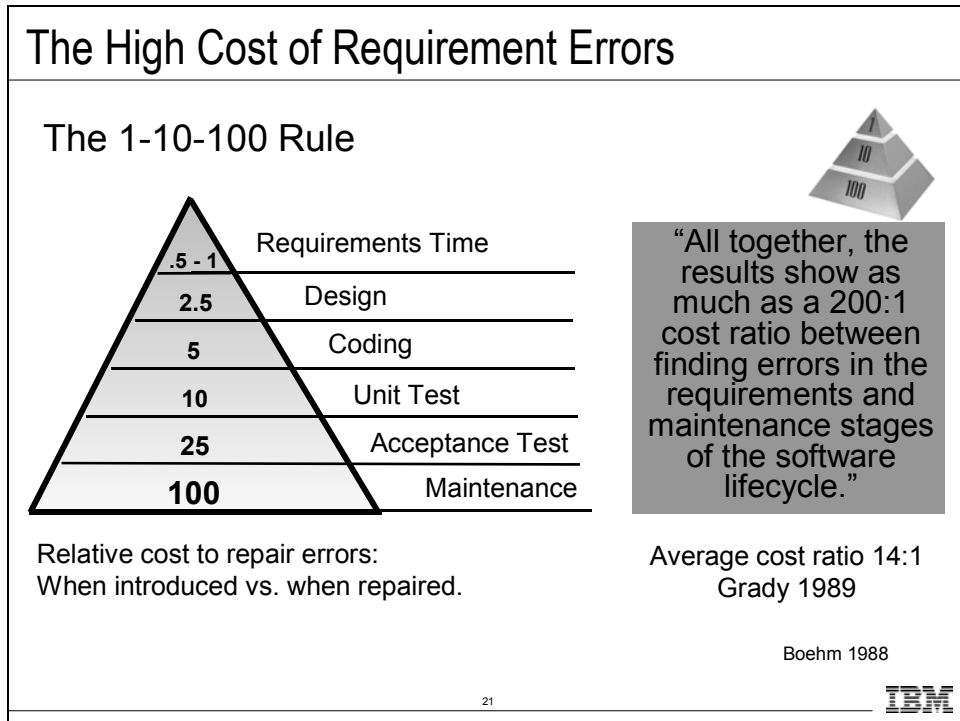
- Smaller projects have smaller teams.
- Smaller projects have fewer requirements.
- Smaller projects have fewer communication issues.
- Smaller projects are easier to manage.
- Smaller projects usually target more focused business objectives.

It is an interesting paradox that, although we know what we are doing wrong, we still continue to make the same mistakes and projects continue to fail.

Does this mean that, if you are on an ambitious project that has a large team and is running over many years you are destined to fail? Absolutely not! But it does mean that if you fail to mitigate the risks these factors present, you will be more likely to fail. Effective requirements management goes a long way to mitigating these risks.

Note: The latest CHAOS report did not provide enough specific data to allow this chart to be updated. While the data are a little old, they still support this statement in the 2001 report: "... This year's CHAOS recipe calls for no more than four people, for no longer than four months at a cost of less than \$500,000. ..."

The High Cost of Requirement Errors



The most expensive errors to correct are those introduced at Requirements Time and not found until after release to the customer. These same errors are also the least expensive to correct if found during Requirements Time. The longer they exist, the more expensive they become.

The figure shows the order-of-magnitude relative costs of fixing a requirement error during requirements capture, coding, and testing.

Note: This formula is based on a waterfall process model. While an iterative process should reduce the cost of requirement errors, it still holds true that the later an error is found in the process, the more expensive it is to fix.

Help Projects Succeed

Help Projects Succeed

- ♦ Problem analysis
 - Understand the problem.
 - Gain stakeholder agreement.
 - Clear statement of business objectives.
- ♦ Requirements elicitation
 - Identify who will use the system (actors).
 - Elicit how the system will be used (use cases).
- ♦ Requirements management
 - Specify requirements completely.
 - Manage expectations, changes, and errors.
 - Control scope creep.
 - Enlist all team members.

22



In order to help projects succeed, pay attention to the factors that the Standish report noted, especially, “Involve users, get complete requirements specifications, and obtain a clear statement of business objectives.”

Do not waste time solving the wrong problem. You need to make sure you understand all the user’s needs.

As you’ll see in the following module, use cases are a way to organize requirements from a user perspective. All of the requirements for a user to accomplish a particular task are gathered together in a single use case. The use-case model is the collection of all the individual use cases.

Because use cases are specified from the user’s perspective, the use-case model is ideal for communicating the proposed system’s functionality and behavior to the customer or user.

How do you know that you are developing the right system? Ask the users and the customer if the use-case model is what they want to do with the system.

The use-case model is also key to developing the right system.

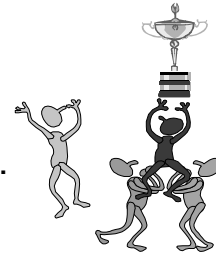
- What do your designers design to? They design a system that allows users to do the tasks specified in the use-case model.
- What do your testers test? They test to make sure that the system is able to perform all the use cases.
- What will the user documentation look like? It documents how to do all the tasks in the use cases.

Could you use help in these areas with your current requirements management process?

Involve the Whole Team in Requirements

Involve the Whole Team in Requirements

- ◆ Developers, Testers, and Writers
 - Help develop requirements management practices.
 - Monitor adherence to practices.
 - Verify elicitation process.
 - Document requirements.
 - Participate in requirements reviews.
 - Participate in or chair a Change Control Board (CCB).
 - Review traceability outcomes.
 - Verify quality, testability, and completeness.



23



What are some of the ways members of different groups of the development team can be involved in your requirements activities?

What are the advantages of involving the whole product development team during requirement activities? The main advantage is that the team gains a better understanding of the requirements and why they are important to the client. What are some other advantages?

Most recommendations regarding standards for the software development process include a strong recommendation to involve the development team in requirement activities. For instance, the recommendation is included in the Capability Maturity Model (CMM).

Value Diminishes as Quality is Compromised



Quality can be measured in many ways. Some examples are: system availability, bugs, and response times.

However, another dimension of quality is the quality of the requirements set. In the next few pages, what makes a quality requirement is discussed.

Meeting the customer's real needs is one of the goals of developing a system. As quality decreases, the ability to meet the customer's needs also decreases. If the system is not meeting the customer's needs, then the perceived value of the system diminishes.

The examples of quality listed above are all observable in the delivered system. They are a bi-product of the quality of the requirements that are implemented. A system can be no better than the requirements used to specify that system.

Qualities of Software Requirements Sets

Qualities of Software Requirements Sets

- ♦ **Correct**
 - Is a true statement of something the system must do.
- ♦ **Complete**
 - Describes all significant requirements of concern to the user.
- ♦ **Consistent**
 - Does not conflict with other requirements.
- ♦ **Unambiguous**
 - Is subject to one and only one interpretation.

Leffingwell & Widrig (1999). IEEE
830-1993, § 4.3.2, 1994

25



Correct

- Does every requirement state something required of the system?
 “A set of requirements is correct if and only if every requirement stated therein represents something required of the system to be built.” *Davis (1993)*
 It is not possible to determine if a requirement is correct simply by reading the requirement. The correctness is verified by a subject-matter expert during a review.

Complete

- Does the set of requirements include all significant requirements, whether related to functionality, performance, design constraints, attributes, or external interfaces?
- Have the expected ranges of input values in all possible scenarios been identified and addressed?
- Have responses been included to both valid and invalid input values?
- Do all figures, tables, and diagrams include full labels, references, and definitions of all terms and units of measure?

Consistent

- Is it internally consistent, with no subset of individual requirements described which are in conflict? (Examples: Vision document, the use-case model, and the Supplementary Specifications.)

Unambiguous

- Does each requirement have one, and only one, interpretation?

Qualities of Software Requirements Sets (cont.)

Qualities of Software Requirements Sets (cont.)

- ♦ Verifiable
 - Can be tested cost effectively.
- ♦ Ranked for importance and stability
 - Can be sorted based on customer importance and stability of the requirement itself.
- ♦ Modifiable
 - Changes do not affect the structure and style of the set.
- ♦ Traceable
 - The origin of each requirement can be found.
- ♦ Understandable
 - Comprehended by users and developers.

Leffingwell & Widrig (1999). IEEE
830-1993, § 4.3.2, 1994

26



Verifiable

- Is every requirement stated verifiable?
- Is there some finite cost-effective process with which a person or machine can check that the software product meets the requirement?

Ability to Rank Requirements

- Has each requirement been tagged with an identifier to indicate either the importance or stability of that particular requirement?

Modifiable

- Are the structure and style of the requirements in the software requirements set (use cases, supplementary specification, or software requirements specification) such that any changes to the requirements can be made easily, completely, and consistently while retaining the structure and style?
- Has redundancy been identified, minimized, and cross-referenced?

Traceable

- Does each requirement have a clear identifier?
- Is it distinguishable from non-essential statements in the requirements set?
- Is the origin of each requirement clear?
- Is backward traceability maintained by explicitly referencing earlier artifacts?
- Is a reasonable amount of forward traceability maintained to artifacts spawned by the requirements set? For example, test cases.

Qualities of a Requirement: Verifiable

Qualities of a Requirement: Verifiable

♦ A requirement is verifiable if:

- There exists some finite, cost-effective process with which a person or machine can check that the product meets the requirement.
- The system supports up to 1,000 simultaneous users.
- The system shall respond to an arbitrary query in 500 msec.
- The color shall be a pleasing shade of green.
- The system shall be available 24 x 7.
- The system shall export view data in comma-separated format, according to the IEEE specification.

Are these requirements verifiable? If not, what is a better way to state them?

IEEE 830-1993, §
4.3.2, 1994

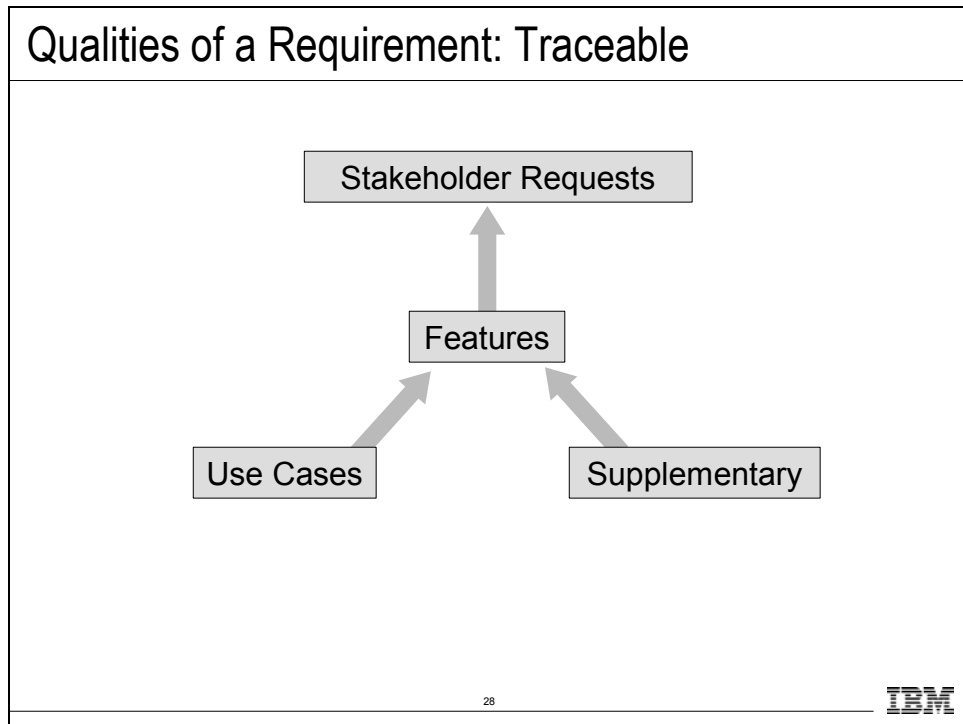
27



Look at each of the above requirement statements and ask, "How can you measure that this requirement has been met?"

Use your glossary to define key terms.

Qualities of a Requirement: Traceable



Traceability (as defined by your RM Plan) allows you to determine the origins of any requirement. Software is expensive and difficult to develop. It is, therefore, important that you only deliver a solution that solves the business problem no more and no less.

Traceability is bi-directional. In the example, you can trace from Stakeholder Requests to Features and vice versa.

A requirement that does not directly fulfill a stakeholder's need is scope creep. The ability to trace the origins of a requirement is an important quality measure.

A stakeholder request that you cannot trace to some implementation means some requirements have been missed.

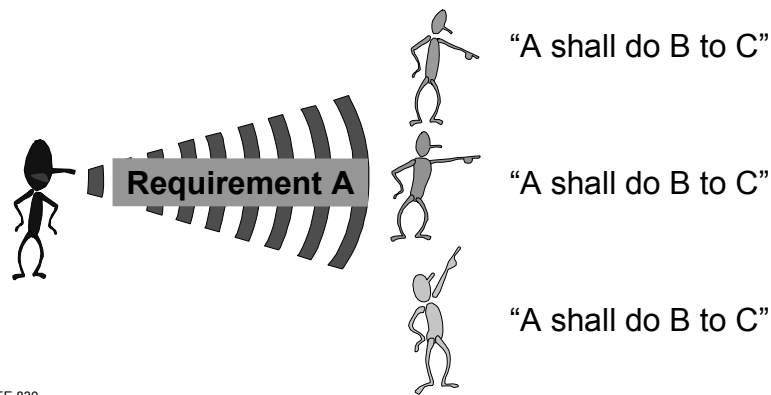
Traceability allows us to:

- Assess the project impact of a change in a requirement.
- Assess the impact of a failure of a test on requirements (that is, if the test fails, the requirement may not be satisfied).
- Manage the scope of the project.
- Verify that all requirements of the system are fulfilled by the implementation.
- Verify that the application does only what it was intended to do.
- Manage change.

Qualities of a Requirement: Unambiguous

Qualities of a Requirement: Unambiguous

- ♦ A requirement is unambiguous if it has only one interpretation.



ref - IEEE 830

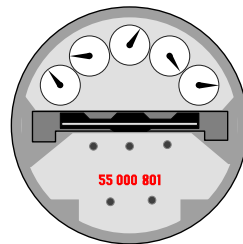
29

IBM

Each stakeholder should have the same interpretation of what a particular requirement means.

Some hints for writing unambiguous requirements are:

- Use natural language for most of the specification.
- Write as clearly and concisely as possible.
- Organize the requirements into use cases so that the context of the requirement is clear. The context often eliminates the ambiguity.
- Use pictures, diagrams, and dialogs to further illustrate the intent and features of the application.



- Augment with formal techniques to fully define the functionality. Always start by describing the requirement using natural language or text. Do not immediately jump into structured or formal techniques unless it is needed for understandability by the readers.
- Use the Glossary.

Exercise: Exploring Ambiguity

Exercise: Exploring Ambiguity

Mary had a little lamb.

In the space below, write (or draw) your detailed understanding of what this sentence means.

Gause & Weinberg, 1989

30



Explore Ambiguity: Dictionary Definitions

Explore Ambiguity: Dictionary Definitions

had - Past of have

have - 1a: To hold in possession as property

4a: To acquire or get possession of: OBTAIN (best to be had)

4c: ACCEPT; to have in marriage

5a: To be marked or characterized by (have red hair)

10a: To hold in a position of disadvantage or certain defeat

10b: TRICK, FOOL (been had by a partner)

12: BEGET, BEAR (have a baby)

13: To partake of (have dinner)

14: BRIBE, SUBORN (can be had for a price)

lamb - 1a: A young sheep esp. less than one year old or without permanent teeth

1b: The young of various other animals (as smaller antelopes)

2a: A person as gentle or weak as a lamb

2b: DEAR, PET

2c: A person easily cheated or deceived especially in trading securities

3a: The flesh of lamb used as food

31

IBM

Two words were picked from the sentence: “had” and “lamb” and look up in the dictionary.

Here are some of the definitions listed.

Now, what if we combined them? What does it do to the meaning of the sentence?

Explore Ambiguity: Analysis

Explore Ambiguity: Analysis

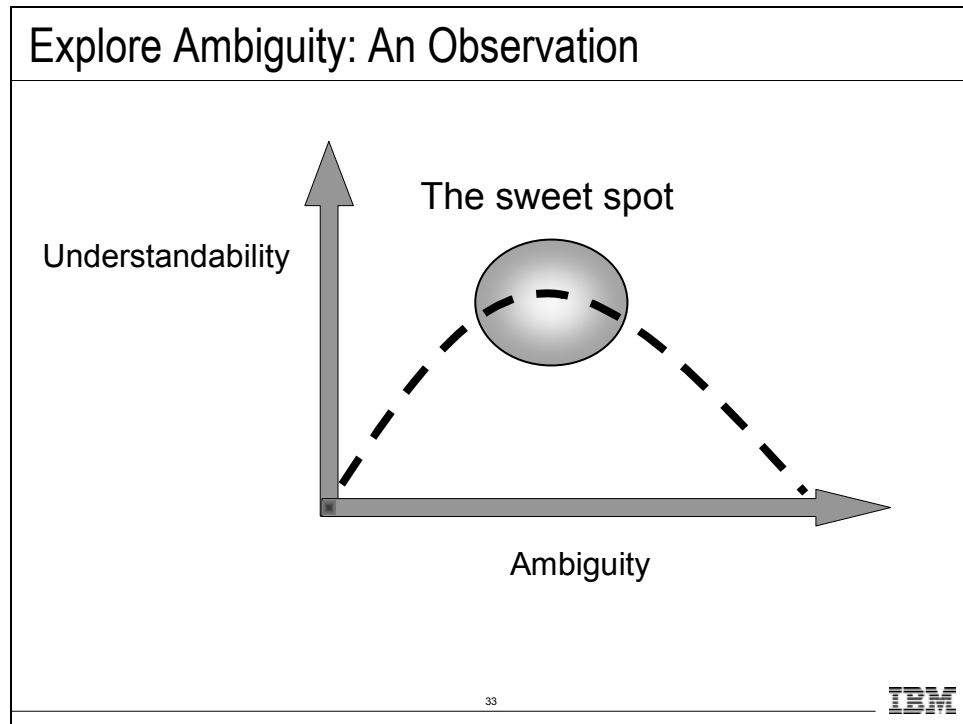
Have	Lamb	Interpretation
1a	1a	Mary owned a little sheep under one year of age or without permanent teeth.
4a	1a	Mary acquired a little sheep under one year of age or without permanent teeth.
5a	1a	Mary is the person who owned a little sheep under one year of age or without permanent teeth.
10a	1a	Mary held a little sheep under one year of age or without permanent teeth in a position of disadvantage.
10b	1a	Mary tricked a little sheep under one year of age or without permanent teeth.
12	1b	Mary gave birth to a young antelope.
12	2a	Mary is (or was) the mother of a particular small, gentle person.
13	3a	Mary ate a little of the flesh of lamb.
14	2c	Mary bribed a small person trading in securities who was easily cheated.

32



By combining the different definitions of the words, you get very different meanings of this apparently simple statement.

Explore Ambiguity: An Observation



Have you ever read a document that was completely unambiguous? Was it understandable?

Techniques that reduce ambiguity in a requirements set often decrease understandability and alienate customers and users. The goal is to find the “sweet spot” where we attain the greatest understandability with the least ambiguity.

To completely remove all ambiguity (the far left of the *Ambiguity* axis) the only way to represent a requirement is with a mathematical equation. However, specifying all requirements with this level of formality would make the requirements set unintelligible to all but the most educated of readers.

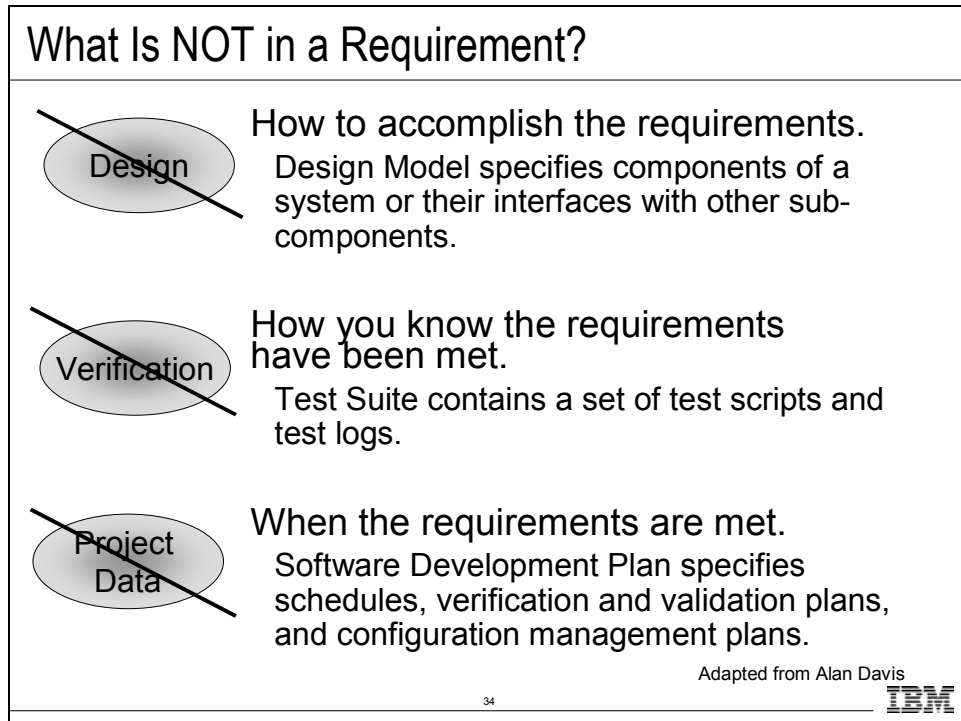
At the other end of the scale you would have a requirement that said: “Build me an onboard trip computer for an automobile”. It is impossible to determine what is required from a requirement stated like this.

The goal is to reduce ambiguity as much as possible, while still increasing understandability. Once you see that by reducing it further, it also decreases understandability, then it’s time to back off. Try to find that apex “sweet spot” in the curve.

The level of ambiguity you should strive for often depends on the cost of getting it wrong. For example, is someone’s life at risk if this requirement is not implemented correctly?

For further information on ambiguity in requirements, refer to Gause and Weinburg, *Exploring Requirements: Quality Before Design*. (1989)

What Is NOT in a Requirement?



The **Design model** is an object model describing the realization of the use cases and serves as an abstraction of the implementation model and its source code. The Design model is used as essential input to activities in implementation and test.

The **Test Suite** is a package-like artifact used to group collections of Test Scripts, both to sequence the execution of the tests and to provide a useful and related set of Test Log information from which Test Results can be determined.

The Test Suite and the requirements set are usually separate documents because they have different authors and audiences. However, some customers, such as the U.S. Military, may require that the requirements set include the Test Suite.

A **Test Script** is the step-by-step instructions that realize a test, enabling its execution. Test Scripts may take the form of either documented textual instructions that are executed manually or computer readable instructions that enable automated test execution.

The **Software Development Plan** is a comprehensive, composite artifact, which gathers all information required to manage the project. It encloses a number of artifacts developed during the Inception phase and maintained throughout the project.

RUP: A Framework for Requirements Management

RUP: A Framework for Requirements Management



35



The Rational Unified Process® or RUP® product is a framework for a software engineering process. It provides a disciplined approach to assigning tasks and responsibilities within a development organization. Its goal is to ensure the production of high-quality software that meets the needs of its end users within a predictable schedule and budget.

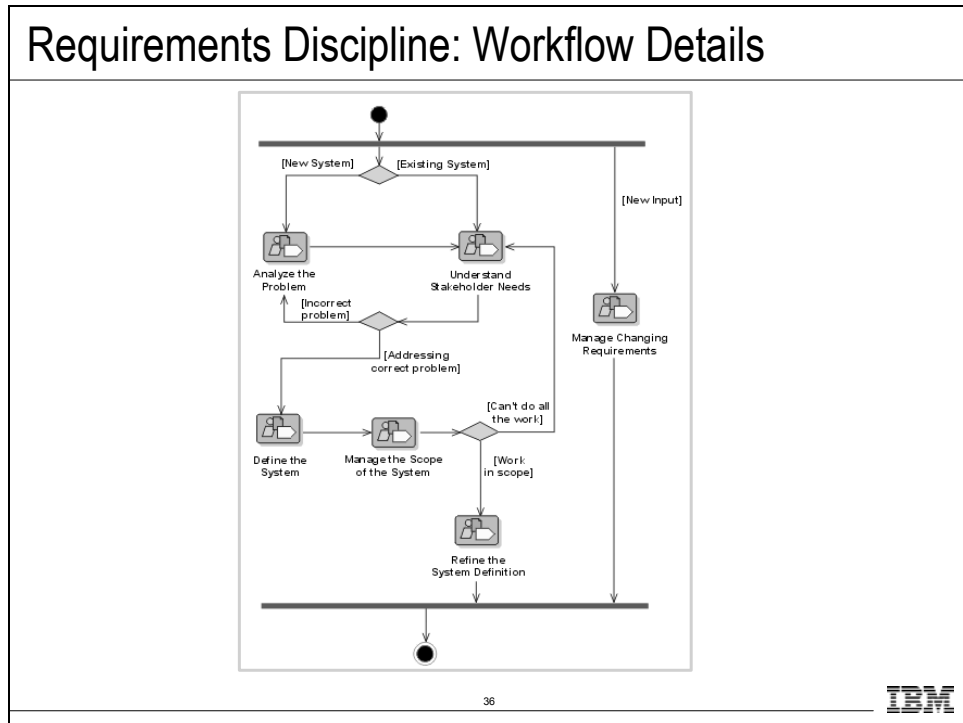
The RUP provides eight disciplines that provide guidance on everything from business modeling to project management.

The purpose of the Requirements discipline is to provide guidance on:

- Establishing and maintaining agreement with the customers and other stakeholders on what the system should do.
- Providing system developers with a better understanding of the system requirements.
- Defining the boundaries of (delimit) the system.
- Providing a basis for planning the technical contents of iterations.
- Providing a basis for estimating cost and time to develop the system.

The RUP is a framework because no two projects are developed the same way. Each project has different development needs. You therefore create a “development case” for each project. This development case describes which activities you perform and when you perform them, and which artifacts you produce.

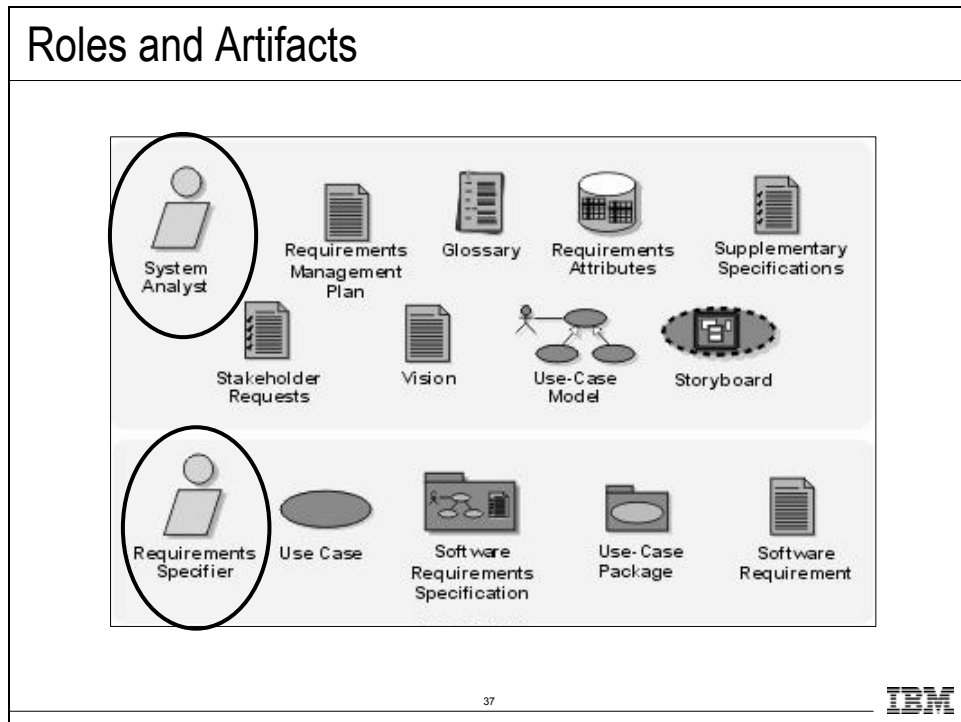
Requirements Discipline: Workflow Details



This diagram shows the Requirements Discipline in the Rational Unified Process®. The major activities in the Requirements discipline are called *workflow details*.

What are the major activities for requirements management in your own software development process?

Roles and Artifacts



In the RUP®, the system analyst role leads and coordinates requirements elicitation and use-case modeling by outlining the system's functionality and delimiting the system. For example, establishing what actors and use cases exist and how they interact.

The requirements specifier role details the specification of a part of the system's functionality by describing the requirements aspect of one or several use cases and other supporting software requirements. The requirements specifier may also be responsible for a use-case package and maintain the integrity of that package. It is recommended that the requirements specifier responsible for a use-case package is also responsible for its contained use cases and actors.

Another important role in the Requirements discipline is the requirements reviewer (not shown here), who plans and conducts the formal review of the use-case model and other requirements.

You will describe these roles and artifacts as you go through the MRMUC course.

What Artifacts Are Used to Manage Requirements?

What Artifacts Are Used to Manage Requirements?		
Where is the problem defined? Where are the stakeholders and users listed? Where are the environments and platforms identified?	Vision	
Where are the non-functional requirements located?	Supplementary Spec	
Where are the use cases maintained?	Use Case Specs	
Where is the common terminology stored?	Glossary	
Where are the stakeholder Needs/Requests captured?	Stakeholder Requests	
38		

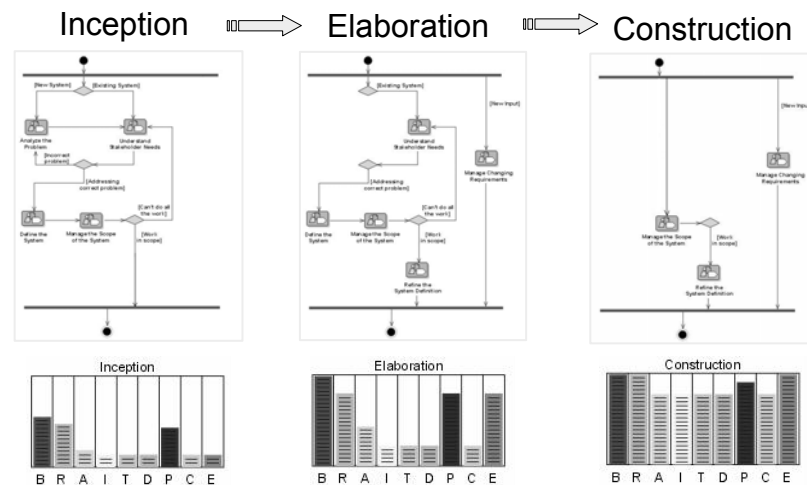
There are a number of different artifacts that you need to work with when managing your software requirements. Above is a brief list to help you visualize what goes where.

Your Requirements Management Plan dictates what artifacts you use to capture your requirements. The RM Plan should be treated as a first-class-citizen and used to guide you through the requirements process.

One Discipline – Many Paths (One Possible Configuration)

One Discipline – Many Paths (One Possible Configuration)

The focus changes from phase to phase and iteration to iteration.



39

IBM

In an iterative process you make a pass through each discipline once per iteration. During each pass you do not invest the same level of effort in each activity. For example, in early Inception iterations, you spend a great deal of effort on problem analysis and little on refining the system definition. In later iterations, you spend less time on problem analysis and much more time on refining the system definition.

Every software project is different, so each should be project managed differently. A small project with little risk does not need the same level of formalism as a large project that has high risk.

The RUP® is not a one-size-fits-all process. It is intended to be customized. Each customization is called a "Development Case." A development case details the activities that are performed and the artifacts that are produced in each discipline for a specific project. The iteration plan describes when each artifact is produced during the project.

Typically, an organization creates a number of standard development cases, for example, one for small low risk projects and another for large medium risk project. As a new project is started it is categorized and a development case is chosen. The development case describes which artifacts are produced and when. The chosen development case may be further customized to make it relevant to the project being undertaken. The idea is that most of the customization work has already been done and only some minor tweaks should be required for the new project.

Review: Introduction to RMUC

Review: Introduction to RMUC

1. What is a requirement?
2. What is requirements management?
3. What factors contribute to project success?
4. What team members are involved in requirements management and how?
5. How would you explain the 1-10-100 rule?
6. Why would you use a development case?
7. What are some requirement characterizations?
8. What are some qualities of requirements?

40



Find answers.

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.
- 8.